

SENG 533 - Concrete Project Plan

Team Members:

Aly Farouz (30169931)
Jung-Hwan Park (30146353)
Chinonso Oragwam (30142426)
Jinsu Kwak (30097737)

Submission Date:

Feb 22, 2026

Project: Performance Comparison of Multiple NoSQL Databases (MongoDB and Cassandra)

Notes about the previous proposal

This concrete proposal will cover the same topic as our initial proposal, which is the performance comparison of MongoDB and Cassandra under controlled workloads. Based on our feedback, we have refined the scope, where we are prioritizing workload-based performance comparison using equivalent workloads and a standardized benchmarking tool.

System Description:

The goal is to analyze the performance of MongoDB and Cassandra under controlled conditions. Both databases will be deployed as single-node instances in Docker containers to make our comparisons reproducible and meaningful.

We will focus on throughput, latency (particularly tail latency at p99 and p95), and resource utilization under different workloads. The goal is to provide evidence based understanding on how these two systems perform under different database operations, so that engineers can make informed decisions when developing software architecture.

Goals/Objectives:

1. **Workload Performance:** Compare MongoDB and Cassandra under read-heavy, write-heavy, and mixed workloads, comparing differences in throughput and latency.
2. **Concurrency & Saturation:** Identify concurrency point at which our system metrics saturate. This occurs when the throughput plateaus and latency spikes.
3. **Resource Bottleneck Analysis:** Identify which system resources that restrict performance under high workload.
4. **Evidence-Based Guidance:** Provide data backed recommendations for engineers on which system best fits a given workload pattern, and ensure it's within project costs.

Past Studies:

Past Studies on NoSQL performance focused on general NoSQL comparisons or single-database benchmarking without separating different factors like concurrency, workload type, or resource utilization.

Abu Kausar et al. (2022) used YCSB to compare MongoDB, Cassandra, and Redis across six workloads with operation counts from 100,000 to 1,000,000. What they found was that overall MongoDB performed best in terms of latency across most workloads, but they neglected metrics like concurrency and resource utilization.

(<https://indjst.org/articles/a-study-of-performance-and-comparison-of-nosql-databases-mongodb-cassandra-and-redis-using-ycsb>)

BenchANT (2022) ran a MongoDB vs Cassandra comparison using YCSB on AWS instances and found that MongoDB performed better on read-heavy workloads while write-heavy workloads favoured Cassandra. One major difference from our upcoming experiment was that their tests used multi-node clusters and did not report tail latency or resource utilization data. While we will not be using multi-node clusters we can expand on tail latency and resource utilization data. (<https://benchant.com/blog/mongodb-vs-cassandra>)

Across these 2 studies and several others, researchers do not report tail latency, saturation points (throughput, concurrency etc), and resource utilization (CPU, Disk Memory). Our project will address these gaps by isolating these factors under controlled single node scenarios.

Important Metrics:

1. Latency: Time delay from request submission to when the client receives a response. We will report average latency as well as P95 Latency and P99 Latency to capture worse case scenarios.
2. Throughput: Evaluate the database's system ability to handle workload demands.
3. Resource Utilization: Evaluate the resource consumption of the database server under different workloads and scenarios. For example, CPU usage, memory consumption, disk I/O of the database container for each run. These will be collected through docker stats and system monitoring tools.

Important Factors:

Factor	Levels
Workload Type	Read-Heavy, Write-Heavy, Mixed
Dataset Size	100 MB, 1 GB, 1 TB
Concurrency	1, 10, 50, 100(threads)
Indexing	Default indexing

Operation Type	Single-key operations, batch operations
----------------	---

Evaluation Technique:

The benchmarking tool we will be using is the Yahoo Cloud Serving Benchmark (YCSB), which has mainly been used to evaluate and compare the performance of NoSQL database systems.

Evaluation Setup:

We will be hosting MongoDB and Cassandra databases on our local systems, and we will use simulation modeling to evaluate the performance of the databases as if they were being used in a real world scenario. MongoDB 7.0, Cassandra 4.1 Python 3.11 and Docker 24.x will be used in this evaluation. The operating systems will be macOS/Windows 10, and the database will run on a single server.

The evaluation will consist of a program that is developed in Python that will allow for different metrics of the MongoDB and Cassandra databases to be extensively tested. The program will allow for certain parameters to be changed such as concurrency levels, data volume, and distribution characteristics that will cause the system to face varying levels of workloads as it would in a real usage situation.

Test Workloads:

We will use the below YCSB-based workloads:

YCSB Workload A (Update heavy): 50% reads, 50% updates. Simulates a session store or user profile cache.

YCSB Workload B (Read heavy): 95% reads, 5% updates. Simulates a heavy content serving application.

YCSB Workload C (Read-modify-write): 50% reads, 50% read-modify-write. Simulates apps that read a record, modify it, and write it back.

Bulk load test: Insert large batches of records to measure write throughput during data loading.

Stress test: From 1 to 100+ threads incrementally increase concurrency until throughput plateaus and latency degrades. This will identify the systems saturation point

Experiment Design:

We use a controlled comparative experiment to evaluate MongoDB and Cassandra under identical conditions. Both databases will run as single-node Docker containers on the same local system, and workloads will be generated using a YCSB-based Python benchmark.

The baseline configuration will be 1 GB dataset, 10 threads, single-key operations.

Workload Series: Run Workloads A, B, C using the same configurations as our baseline workload.

Concurrency Series: Run baseline workload at 1, 10, 50, 100 threads.

Dataset Size Series: Run baseline workload with 100MB and 1GB datasets.

Operation type Series: Compare single-key lookups, scans, and batch inserts.

For each test, we follow the same process: initialize database → load data → warm-up → run the workload → record all metrics → reset the database. Each scenario will be repeated at least 3 times and results will be compared, making sure to take into account averages and standard deviations to take into account variability.

This gives us approximately 2 (databases) x 12-15 (configurations) x 3 (repetitions) = 72-90 individual test runs.

This design ensures a fair, reproducible comparison of both systems under equivalent workloads, while ensuring that results reflect single-node performance rather than full distributed cluster behavior.

Analysis Plan

We will analyze our results using the below methods:

Baseline Comparison:

After establishing a baseline performance for both databases using the default configuration. Compare all changes in configuration to this baseline.

Statistical Summary:

For each configuration record stats like mean, standard deviation, P95 and P99 latency. This will help us determine if our results are meaningful or noise.

Visualization:

We will create charts comparing throughput vs concurrency, latency, and resource utilization over time. These charts will be created using python matplotlib or seaborn

Saturation Points:

By plotting throughput and latency and then relating it resource utilization we can determine saturation points and resource bottle necks.

Final Summary:

Finally we will summarize all our findings in a table that is easy to digest showing the contributions of each configuration parameter.

Timeline:

Date	Milestone
February 22nd	Complete concrete project proposal
February 25	Set up Docker containers for MongoDB and Cassandra. Make sure it works across all our systems.
March 2nd	Set up docker with Cassandra and MongoDB
March 5th	Complete baseline tests for both databases
March 9th	Set up benchmarking (YCSB)
March 15th	Midterm report submission (Our baseline + initial results)
March 22	Complete dataset size and operation type test series
April 1st	Complete experiment execution
April 7th	Make data visualizations and analysis
April 10th	Complete final report